

From Ontologies to Heterogeneous Quantum Backends: A QA-Prolog Pipeline toward HPC Workflows for AI

Davide Camino and Luca Roversi



UNIVERSITÀ
DI TORINO

$$\langle AIQ | \otimes | QIA \rangle$$

Perugia, 07/10/2026

Goal

Enabling ontology reasoning on quantum hardware

Why?

- Ontologies are still relevant
- Fill gap between high-level symbolic representations and quantum backend
- Broader diffusion of quantum computing

How?

- Modular
- Scalable
- Inspectable

Rewriting Pipeline

Performing Fully Parallel Constraint Logic Programming on a Quantum Annealer

SCOTT PAKIN

Computer, Computational, and Statistical Sciences Division
Los Alamos National Laboratory
MS B287
Los Alamos, New Mexico, USA
E-mail: pakin@lanl.gov

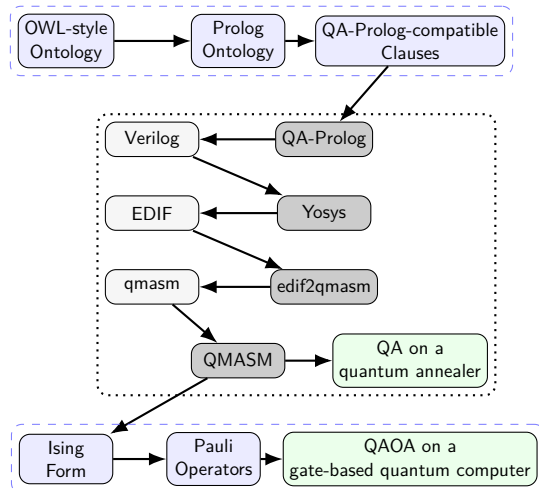
submitted 31 March 2017; revised 11 January 2018; accepted ???

Abstract

A *quantum annealer* exploits quantum effects to solve a particular type of optimization problem. The advantage of this specialized hardware is that it effectively considers all possible solutions in parallel, thereby potentially outperforming classical computing systems. However, despite quantum annealers having recently become commercially available, there are relatively few high-level programming models that target these devices.

In this article, we show how to compile a subset of Prolog enhanced with support for constraint logic programming into a 2-local Ising-model Hamiltonian suitable for execution on a quantum annealer. In particular, we describe the series of transformations one can apply to convert constraint logic programs expressed in Prolog into an executable form that bears virtually no resemblance to a classical machine model yet that evaluates the specified constraints in a fully parallel manner. We evaluate our efforts on a 1095-qubit D-Wave 2X quantum annealer and describe the approach's associated capabilities and shortcomings.

KEYWORDS: quantum annealing, quantum computing, constraint logic programming, Prolog, D-Wave



Rewriting Pipeline

Performing Fully Parallel Constraint Logic Programming on a Quantum Annealer

SCOTT PAKIN

Computer, Computational, and Statistical Sciences Division
Los Alamos National Laboratory
MS B287
Los Alamos, New Mexico, USA
E-mail: pakin@lanl.gov

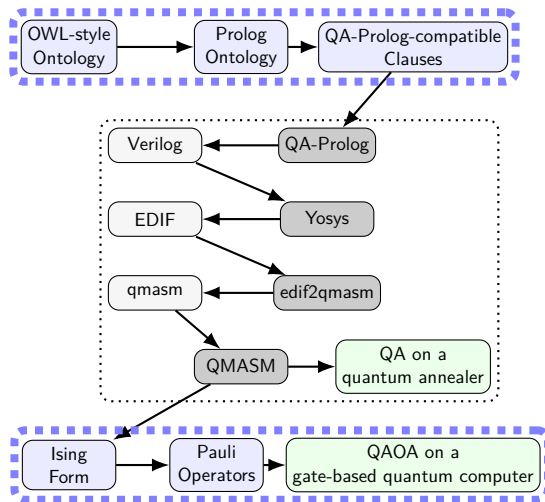
submitted 31 March 2017; revised 11 January 2018; accepted ???

Abstract

A *quantum annealer* exploits quantum effects to solve a particular type of optimization problem. The advantage of this specialized hardware is that it effectively considers all possible solutions in parallel, thereby potentially outperforming classical computing systems. However, despite quantum annealers having recently become commercially available, there are relatively few high-level programming models that target these devices.

In this article, we show how to compile a subset of Prolog enhanced with support for constraint logic programming into a 2-local Ising-model Hamiltonian suitable for execution on a quantum annealer. In particular, we describe the series of transformations one can apply to convert constraint logic programs expressed in Prolog into an executable form that bears virtually no resemblance to a classical machine model yet that evaluates the specified constraints in a fully parallel manner. We evaluate our efforts on a 1095-qubit D-Wave 2X quantum annealer and describe the approach's associated capabilities and shortcomings.

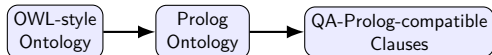
KEYWORDS: quantum annealing, quantum computing, constraint logic programming, Prolog, D-Wave



Extensions

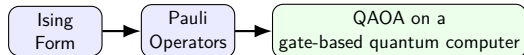
Up-Stream Extensions

- OWL to Prolog
- Prolog to QA-Prolog
- Preliminary study of overhead



Down-Stream Extensions

- Output of QA-Prolog encoded in Qiskit
- Results' Evaluation
- Preliminary tests on improvement



Extensions

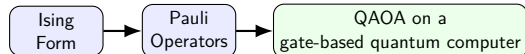
Ontology to QA-Prolog

- OWL to Prolog
- Prolog to QA-Prolog
- Preliminary study of overhead



QUBO as portable representation

- Output of QA-Prolog encoded in Qiskit
- Results' Evaluation
- Preliminary tests on improvement



Results

Restored pipeline available for everyone to experiment with

Up-Stream results

- 1 Ontology encoded in QA-Prolog
- 2 Proposed solution to incompatibility
- 3 Preliminary study of problem's dimension

Down-Stream results

- 1 Proof of portability
- 2 Statistical evaluation of QAOA results
- 3 Initial ideas on improvement of results

Results

Restored pipeline available for everyone to experiment with

Up-Stream results

- 1 Ontology **encoded** in QA-Prolog
- 2 Proposed solution to **incompatibility**
- 3 Preliminary study of **problem's dimension**

Down-Stream results

- 1 Proof of **portability**
- 2 Statistical evaluation of **QAOA** results
- 3 Initial ideas on **improvement of results**

Results

Restored pipeline available for everyone to experiment with

Up-Stream results

- 1 Ontology **encoded** in QA-Prolog
- 2 Proposed solution to **incompatibility**
- 3 Preliminary study of **problem's dimension**

Down-Stream results

- 1 Proof of **portability**
- 2 Statistical evaluation of **QAOA** results
- 3 Initial ideas on **improvement of results**

Achieved semantic reasoning on quantum hardware

Ontology

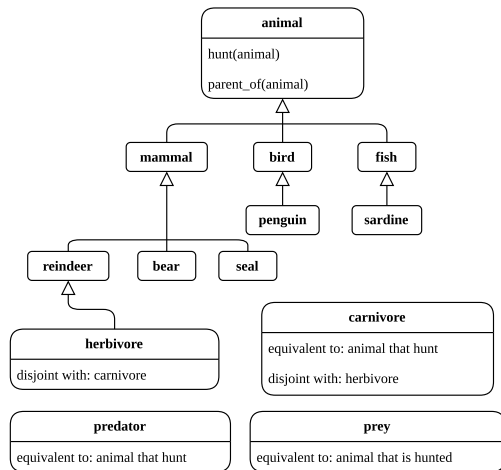


Figure: T-Box

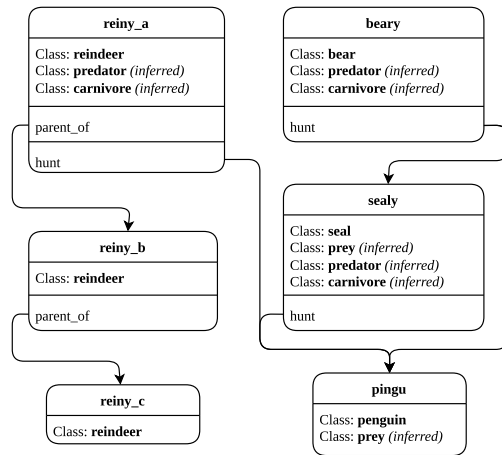


Figure: A-Box

OWL → Prolog

Translating OWL and Semantic Web Rules into Prolog: Moving Toward Description Logic Programs

Ken Samuel¹, Leo Obrst¹,
Suzette Stoutenberg², Karen Fox², Paul Franklin², Adrian Johnson²,
Ken Laskey¹, Deborah Nichols¹, Steve Lopez², Jason Peterson²
{samuel, lobrst, suzette, kfox, pfranklin, abjohnson,
klaskey, dlnichols, slopez, jasonp}@mitre.org

The MITRE Corporation

¹ 7525 Colshire Drive, McLean, VA 22102-7508

² 1155 Academy Park Loop, Colorado Springs, CO 80910-3716

Submitted 12 May 2006; revised 24 Sep 2007; accepted 18 October 2007

Abstract. We are researching the interaction between the rule and the ontology layers of the Semantic Web, by comparing two options: 1) using OWL and its rule extension SWRL to develop an integrated ontology/rule language, and 2) layering rules on top of an ontology with RuleML and OWL. Toward this end, we are developing the SWORIER system, which enables efficient automated reasoning on ontologies and rules, by translating all of them into Prolog and adding a set of general rules that properly capture the semantics of OWL. We have also enabled the user to make dynamic changes on the fly, at run time. This work addresses several of the concerns expressed in previous work, such as negation, complementary classes, disjunctive heads, and cardinality, and it discusses alternative approaches for dealing with inconsistencies in the knowledge base. In addition, for efficiency, we implemented techniques called extensionalization, avoiding reanalysis, and code minimization.

Keywords: Semantic Web, logic programming, knowledge compilation, ontologies, rules

Implication of disjointedness

```
1  logicNot(isA(I, C)) :-  
2      disjointClasses(C, C2),  
3      isA(I, C2).
```

Definition of subclass

```
1  subclass(C1, C2) :-  
2      subclass(C1, C2).  
3  subclass(C1, C2) :-  
4      subclass(C1, C3),  
5      subclass(C3, C2).
```

Prolog → QA-Prolog

No Complex Predicates

```
1  logicNot(isA(I, C)) :-  
2      disjointClasses(C, C2),  
3      isA(I, C2).
```

→

```
1  isNotA(I, C) :-  
2      disjointClasses(C, C2),  
3      isA(I, C2).
```

No Recursion

```
1  subClass(C1, C2) :-  
2      subclass(C1, C2).  
3  subClass(C1, C2) :-  
4      subclass(C1, C3),  
5      subClass(C3, C2).
```

→

```
1  subClass0(C1, C2) :-  
2      subclass(C1, C2).  
3  subClass1(C1, C2) :-  
4      subclass(C1, C3),  
5      subClass0(C3, C2).  
6  subClass2(C1, C2) :-  
7      subclass(C1, C3),  
8      subClass1(C3, C2).
```

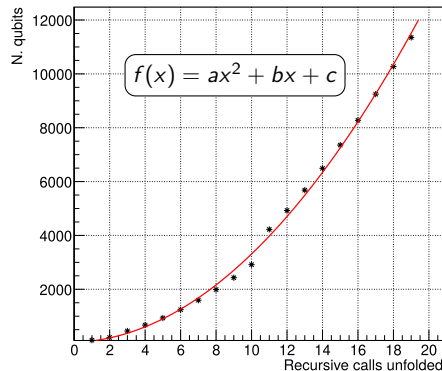
Impact of Unfolding

Dimension of the problem

- N. qubits to represent the QUBO
- Dimension of the matrix

Causes of overhead

- New rule/module each level
- Final big OR
- Unique name for each rule



	<i>c</i>	<i>b</i>	<i>a</i>	<i>R</i> ²	<i>p</i> _{model}
Estimate	112.78	-71.49	32.13	0.997	2.28×10^{-21}
Std. Error	187.59	38.79	1.72	—	—
<i>p</i> -value	0.556	0.084	< 0.001	—	—

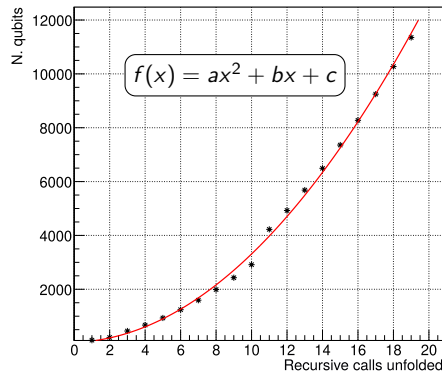
Impact of Unfolding

Dimension of the problem

- N. qubits to represent the QUBO
- Dimension of the matrix

Causes of overhead

- New rule/module each level
- Final big OR
- Unique name for each rule



	c	b	a	R ²	p _{model}
Estimate	112.78	-71.49	32.13	0.997	2.28 × 10 ⁻²¹
Std. Error	187.59	38.79	1.72	—	—
p-value	0.556	0.084	< 0.001	—	—

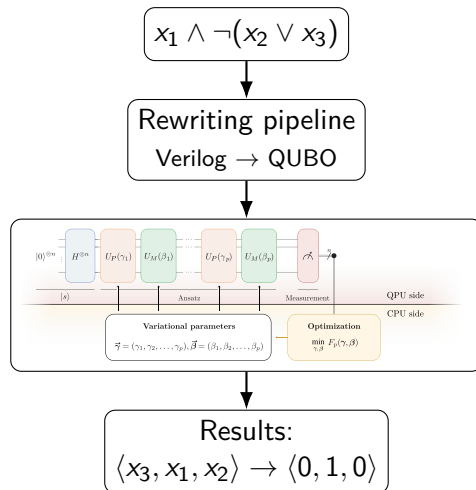
QAOA

Goal

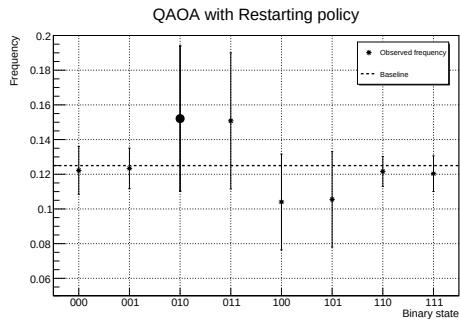
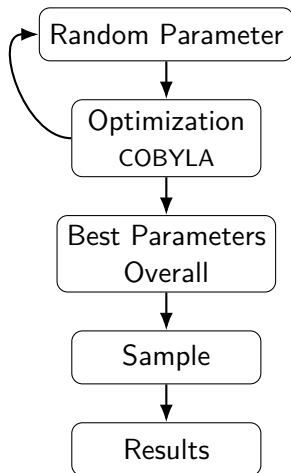
- Vendor Independent
- Paradigm Independent
 - Quantum Annealing,
 - Gate Based,
 - Simulation, ...

Preliminary inspection

- Simple problem
- Proving portability
- Ideas on how to improve



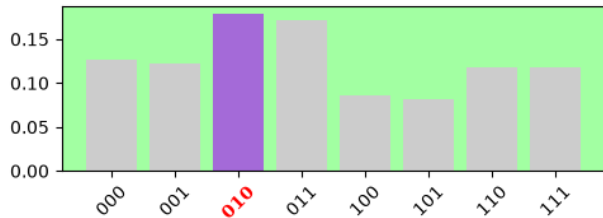
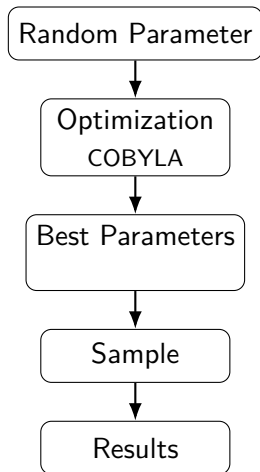
Restarting Policy



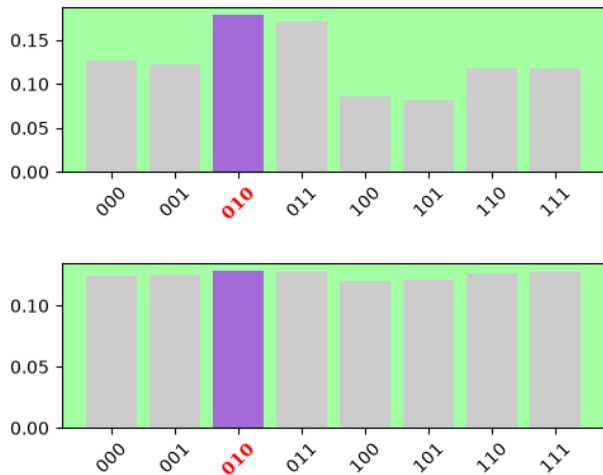
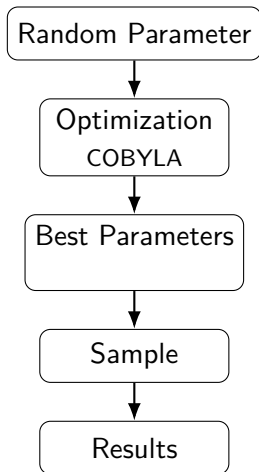
Results

- High reproducibility
- Probability of right assignment above baseline
- Big error bar

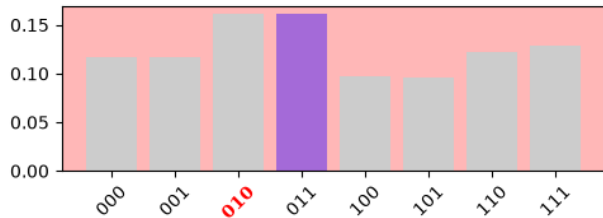
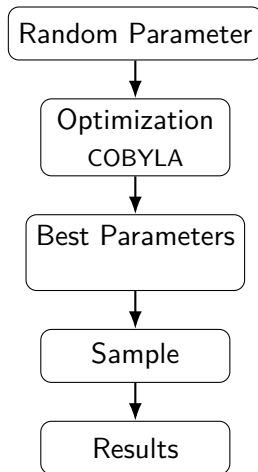
Naïf Implementation



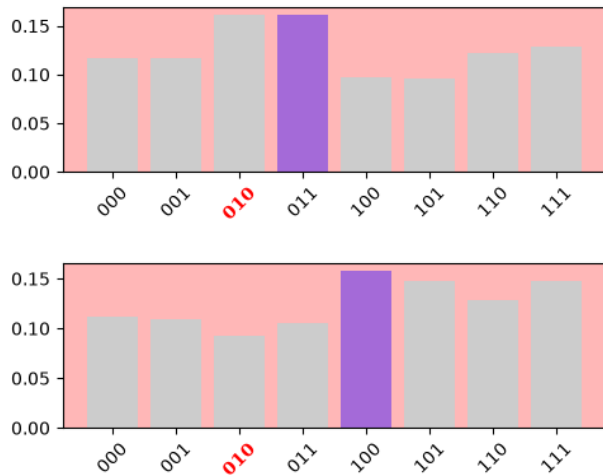
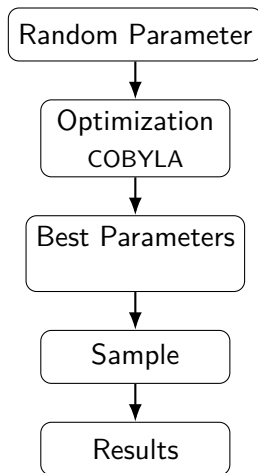
Naïf Implementation



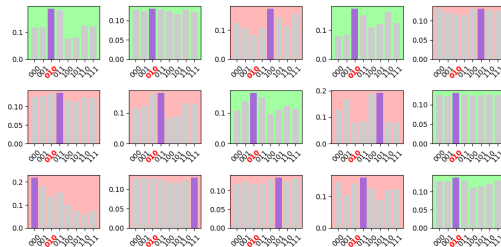
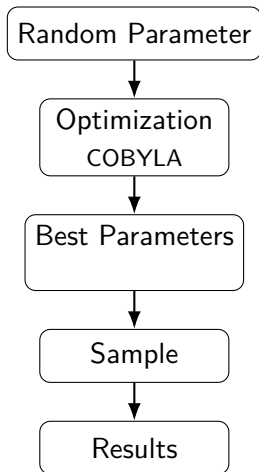
Naïf Implementation



Naïf Implementation



Naïf Implementation



Pro

- Correct assignment 6 times out of 15

Cons

- High variability of results (low reproducibility)
- Quite flat distribution

Study Limitation

Preliminary exploration

- Small ontology
- Quantum execution only simulated
- Not end to end QAOA execution

Future Works

Pipeline Compatibility

- Integrate the pipeline in hybrid workflows (QSPLIT & StreamFlow)
- Interface the pipeline with other solvers to exploit the results' high level interpretation
- Start the pipeline from LLVM

Results' improvement

- Characterization of logic gates
- Basis of logic gates that makes QAOA effective
- Modify the definition of Hamiltonian associated to logic gates
- Explore different parameter optimization algorithms

Thanks for your Attention



Thanks for your Attention



Any questions?

For longer discussions, the slides and other:
davide.camino@edu.unito.it

Inconsistency detection

No Complex Predicates

```
1  logicNot(isA(I, C)) :-  
2      disjointClasses(C, C2),  
3      isA(I, C2).
```

→

```
1  isNotA(I, C) :-  
2      disjointClasses(C, C2),  
3      isA(I, C2).
```

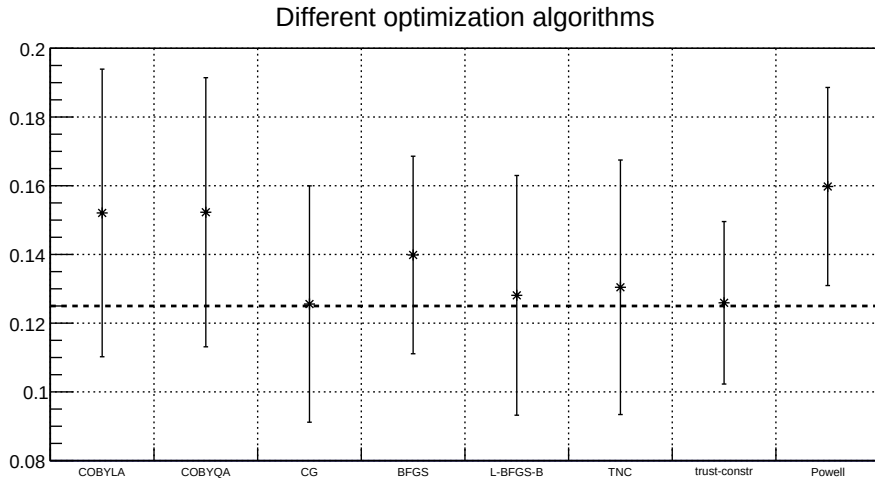
Prolog error

```
1  error(['A term cannot be true and false.', P]) :- logicNot(P), P.
```

QA-Prolog error

```
1  error(I, C, M) :-  
2      isA(I, C),  
3      isNotA(I, C),  
4      M = 'Individual X cannot belong and not belong to class Y'.
```

Different Optimization Algorithms



Dilated Hamiltonian

